Android-Projekte – GitHub-Builds, Updates und Bestandsschutz – v2.0

Projektübergreifende Basisrichtlinie mit verbindlichem Prüfschema und projektspezifischem Anhang.

Feld	Festlegung
Geltungsbereich	Android-Apps mit APK-Verteilung über GitHub oder gleichwertige CI-Systeme
Dokumentart / Version	Projektvorgabe / v2.0
Datum / Status	14.10.2026 / Entwurf zur fachlichen Freigabe
Verantwortung	Projektverantwortung je Projekt festzulegen; Ausarbeitung: ChatGPT
Ablage	<Projekt>/DOCUMENTATION/Update_Bestandsschutz/ (Vorschlag)
Referenz	PotensicProxy_TAF-Projektvorgabe v1.1 und Änderungsauftrag GitHub-Updates v0.2

Diese Richtlinie definiert Soll-Anforderungen. Sie ist kein Nachweis einer Implementierung, erfolgreichen Installation oder eines Gerätetests.

1. Zweck, Geltung und Ebenen

Ziel ist ein reproduzierbarer und sicherer Build- und Updateprozess, bei dem Berechtigungen, Freigaben, Einstellungen und Nutzerdaten geschützt bleiben. Generische Anforderungen gelten projektübergreifend; App-spezifische Sicherheits- und Buildregeln sind in einem versionierten Projektanhang festzulegen. Abweichungen sind vor der Implementierung schriftlich zu begründen.

2. Voraussetzungen in GitHub und Build-Infrastruktur

- Repository-Zugriffsmodell dokumentieren: öffentlich oder privat; bei privaten Repositories keine dauerhaft gültigen Zugriffstokens in einer APK hinterlegen.
- CI-Workflow mit eindeutigem Trigger, Buildkanal, gesicherter Toolchain und nachvollziehbaren Logs; Build-Caches dürfen Korrektheit und Funktionsumfang nicht verändern.
- Lockfiles und Buildwerkzeugversionen kontrolliert führen; vollständigen Build ohne Cache als Referenzprüfung ermöglichen.
- Debug- und Release-Signierung bewusst trennen; für fortlaufende In-Place-Updates pro Installationskanal eine kompatible, langfristig verfügbare Signatur sicherstellen.
- Keystore und Passwörter ausschließlich in geschützter Secret-Verwaltung; keine Werte in Quellpaketen, Logs oder Artefakten. Schlüsselwiederherstellung separat dokumentieren.
- Für veröffentlichte Updates: eindeutiger versionCode, Paketkennung, überprüfte Signatur, APK, Release-Notes, Prüfsumme und verlässliche Update-Metadaten bereitstellen.
- Test-Builds und reguläre Releases getrennt kennzeichnen; unvollständige oder fehlgeschlagene Builds nicht als aktuelle Updates ausweisen.

3. Voraussetzungen innerhalb der Android-App

- Updatekanal und Updatequelle explizit konfigurieren; Versionen numerisch anhand von versionCode bzw. vertrauenswürdigen Manifestfeldern vergleichen.
- Manuelle Prüfung ermöglichen; automatische Prüfung und optional automatischen Download einstellbar machen, mit sinnvollen Intervallen und Offline-/Rate-Limit-Behandlung.
- APK über HTTPS laden, Downloadfortschritt, Status und verständliche Fehlermeldungen darstellen und einen Downloadabbruch anbieten.
- Paketname, Versionscode, Signatur und Integrität vor Übergabe prüfen; SHA-256 aus einer authentisch bezogenen Quelle verwenden.
- Installation ausschließlich über unterstützte Android-Schnittstellen und notwendige Systembestätigung; bei unklaren Berechtigungen verständlich informieren.
- Vor Installationsübergabe erneut den projektspezifischen Sicherheitszustand prüfen; unsichere, unbekannte oder kritische Zustände sperren.
- Nach Neustart Ergebnis sowie Daten- und Berechtigungsbestand verifizieren; keine stille Datenlöschung oder unkontrollierte Neufreigabe.

4. Bestandsschutz für Daten und Berechtigungen

Ein kompatibles Update muss als In-Place-Update erfolgen. Eine Deinstallation mit Datenverlust darf nicht als reguläres Update ausgegeben werden. Zu bewahren und zu prüfen sind, soweit die Anwendung sie verwendet:

- Android-Laufzeitberechtigungen, dauerhaft gewährte SAF-/URI-Zugriffe und sonstige dokumentierte Zugriffsfreigaben.
- Einstellungen, lokale Konfigurationen, geschützte Anmeldedaten und gespeicherte Verbindungen.
- Benutzerdokumente, Bilder, Metadaten, lokale Datenbanken, Sitzungen, Modelle und Auswertungsresultate.
- Datenformat- und Schema-Kompatibilität; Migrationen müssen getestet und bei Rückwärtsänderungen entsprechend abgesichert sein.

Wichtig: Android erhält Berechtigungen und App-Daten bei kompatiblen Updates typischerweise, aber nicht unter allen Umständen. Deshalb wird Erhalt nicht nur behauptet, sondern vor und nach jedem Update überprüft. Verlust oder Widerruf von Rechten muss gezielt behandelt werden.

5. Betriebs- und Sicherheitsfreigaben

Dauerhafte Android-Zugriffsrechte sind von flüchtigen Betriebs- und Sicherheitsfreigaben zu unterscheiden. Letztere sind nach App-Neustart oder Update aus verlässlichen aktuellen Zuständen neu zu bestimmen. Eine zuvor gültige Sicherheitsfreigabe darf nicht ungeprüft übernommen werden.

Das jeweilige Projekt definiert eine nachprüfbare Installationssperre (z. B. aktiver Flug, laufender Maschinenzyklus, kritische Verbindung, unzureichender Akkustand). Unbekannte sicherheitsrelevante Zustände sind gesperrt, nicht als frei zu interpretieren.

6. Unterbrechung, Fehlerbehandlung und Rückkehr

- Suche und Download sind abbrechbar; bis zur Übergabe an den Android-Installer bleibt die bestehende App weiter nutzbar.
- Eine bereits laufende Systeminstallation ist nicht in jeder Phase kontrollierbar oder abbrechbar.
- Ein In-Place-Downgrade mit niedrigerem versionCode und Erhalt aller App-Daten ist bei normalen Android-Apps nicht allgemein garantiert. Eine „jederzeitige Rückkehr“ darf daher nicht versprochen werden.

- Vorversions-APK, Versionshistorie, Datenmigrationen, Backups und Wiederherstellungsweg dokumentieren; vor Rollback Kompatibilität und Signatur prüfen.
- Nach Fehlversuch Ausgangszustand soweit technisch möglich bewahren, verständlichen Fehler anzeigen und die nächste sichere Handlung anbieten.

7. Verbindliches Prüfschema je Update

Phase	Nachweis
Vorher	Installierte Version, Signatur, relevante Berechtigungen/SAF-Zugriffe und Dateninventar ohne vertrauliche Log-Inhalte prüfen
Download	Quelle, Versionsvergleich, Integrität, Fortschritt und Abbruch testen
Vor Installation	Kompatibilität, projektspezifische Sicherheitsfreigabe und Schwellenwerte erneut prüfen
Danach	Versionscode, App-Start, Berechtigungen, Zugriffe, Nutzerdaten, Einstellungen und Kernfunktionen abgleichen
Fehlerfall	Abbruch, Offline, Speichermangel, beschädigte APK, Signaturabweichung, Migration und gegebenenfalls Wiederherstellung prüfen

8. Dokumentation und Abnahme

Jedes Projekt führt einen dauerhaften Anhang mit Repository, Branch, Build-Triggern, Kanälen, Paketkennung, Signaturstrategie, Sicherheitsvorgaben, Datenorten und Testfällen. Bei jeder Auslieferung: vollständige Projektdateien, Änderungspaket in Originalstruktur, gegebenenfalls Patch, englische Commit-Nachricht mit Datum und Dateiliste sowie inhaltlich deckungsgleiche PDF-/DOCX-Dokumentation nach den jeweiligen Projektregeln. Abnahmestatus und offene Risiken müssen ausdrücklich ausgewiesen sein.

Anhang A – PotensicProxy_TAF (projektspezifische Festlegungen)

Kennung	Bedeutung	Wirkung
BL_	Build Large	Debug-Build ohne Cache; Artefakt für manuelle Tests
BS_	Build Small	Debug-Build mit gültigem Cache; funktional identisch
BA_	Build Automatic Update	Updatefähige Debug-APK mit festem Signierschlüssel, steigendem versionCode und GitHub-Prerelease

Die drei Kennungen verändern nicht den Funktionsumfang. Für BA_ sind die signaturkompatible Vorinstallation, GitHub-Update-Metadaten und eine erneute Freigabe vor Installation erforderlich. Die bisherige allgemeine Commit-Präfixregel BL_/BS_ ist für PotensicProxy_TAF ausdrücklich um BA_ zu erweitern.

PotensicProxy-spezifisch gilt: keine Installation bei aktivem oder ungeklärtem Flugbetrieb, laufendem PStart/PLand oder sicherheitsrelevanter Verbindung; Handy-Akku > 40 %. Vorhandene SAF-Ordnerfreigaben, PStart-Sitzungen samt Bild-/XMP-Daten sowie PPort-Modelle und Einstellungen sind zu erhalten. Flug- und

PLand-Sicherheitsfreigaben sind nach dem Update neu zu bewerten. Ob zusätzlich der Drohnenakku > 40 % nachzuweisen ist, bleibt projektspezifisch zu entscheiden.

Anhangstatus: fachliche Soll-Festlegung; Implementierungs- und Gerätetestnachweise sind releasebezogen zu führen.

Änderungshistorie

Version	Datum	Status	Änderung
v1.0	14.10.2026	Entwurf	PotensicProxy-Bestandsschutz und Wiederherstellung
v1.1	14.10.2026	Entwurf	Buildkennungen BL_/BS_/BA_ ergänzt
v2.0	14.10.2026	Entwurf	Projektübergreifende Richtlinie und separater PotensicProxy-Anhang